

IAM AND SECRETS · CI/CD · OBSERVABILITY · INCIDENT RESPONSE

# Courier checkpoint 2

## *Problem statement*

Two hours · individual · your own sandbox project · Cloud Shell and the console, no code

Version 1.0 · September 2026 · Cloud Shell gcloud 583.x · region asia-south1

# Contents

|                                      |   |
|--------------------------------------|---|
| The brief.....                       | 3 |
| Rules and environment.....           | 4 |
| Task 0 — Baseline (not graded).....  | 5 |
| Task 1 — Secrets and identities..... | 6 |
| Task 2 — Pipeline and promotion..... | 7 |
| Task 3 — Signals.....                | 8 |
| Submission.....                      | 9 |

## The brief

Courier is back. In the first checkpoint you packaged and deployed its two services; they have been running since. The team now wants them run properly: the API key kept in a managed secret and rotated; every change to parcel-api built through a gated pipeline and promoted staging → production behind an approval and a canary; and the service watched, with a log-based metric on its own fault line, an alert that reaches a person, and a check on the public page.

| Service                   | What it does                                                                                                                                                                                                          | Who may call it                                                      |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| <b>parcel-api 2.0</b>     | JSON API from checkpoint 1 plus GET /config (fingerprints of the secret as the container sees it — never the value) and a FAULT_RATE setting that makes /parcels fail on purpose, logging the line "simulated fault". | Only track-web (its own identity, ID token). Nobody on the internet. |
| <b>track-web 1.0</b>      | The public tracking page. Unchanged.                                                                                                                                                                                  | Anyone.                                                              |
| <b>parcel-api-staging</b> | Created by your pipeline in Task 2 — the same image, a second service.                                                                                                                                                | Only you.                                                            |

Both services are supplied complete with their Dockerfiles. You will not write or edit a line of code, a Dockerfile, a test or a pipeline file: everything you produce is configuration and operations — a secret and four bindings, a pipeline run and a release, and a metric, a policy and a check. All of it is verified by reading your project, so leave everything running at the end.

## What you hand in

- SUBMISSION.md from the kit, every blank filled with the real value from your project.
- Your project, left running: the instructor grades it with a script that reads live state (secrets, bindings, builds, releases, revisions, policies).
- Nothing else. No screenshots, no pasted logs.

## How you are assessed

100 points, pass mark 70. Every point comes from an automated check of your project against the acceptance criteria in each task — names matter, so use the ones given. Partial work scores partial points; a task you skip scores nothing, and Task 2 depends on Task 1 being complete.

# Rules and environment

## Rules

- Individual work in your own sandbox project. Talking is fine; sharing commands is not.
- Cloud Shell and the console only. Do not edit anything under `services/`, `tests/`, `deploy/` or `cloudbuild.yaml` — the sed in Task 2 that fills in your project id is the only change to a kit file.
- Open book: [docs.cloud.google.com](https://docs.cloud.google.com), the four session lab manuals and your own notes.
- Time-box: 2 hours from the moment the kit is released. Submit what you have at the end.
- Fixed names, because the grader looks for them: secret `courier-api-key`; service accounts `parcel-api-sa`, `track-web-sa`, `courier-deployer`; pipeline `parcel-api` with targets `staging` and `prod`; metric `parcel_api_faults`; services `parcel-api`, `parcel-api-staging`, `track-web`; region `asia-south1`.

## Environment

| Item           | Value                                                                                                                                    |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Project        | Your sandbox project (Owner role), billing attached by the programme                                                                     |
| Shell          | Cloud Shell — <code>gcloud</code> , <code>git</code> , <code>curl</code> , <code>python3</code> pre-installed                            |
| Region         | <code>asia-south1</code> for everything                                                                                                  |
| Kit            | <code>gcp-checkpoint2-starter.zip</code> → <code>courier-checkpoint2/</code> (see <code>README.md</code> inside)                         |
| Starting state | Task 0's <code>baseline.sh</code> builds it for you: registry <code>courier</code> , both services, both identities, the invoker binding |

### CLOUD SHELL

```
$ unzip -q gcp-checkpoint2-starter.zip && cd courier-checkpoint2 && ls
README.md SUBMISSION.md baseline.sh cloudbuild.yaml deploy observability
pyproject.toml requirements-dev.txt services tests
```

Variables used in every task below — set them once per Cloud Shell session:

```
PROJECT=$(gcloud config get-value project); R=asia-south1;
AR=$R-docker.pkg.dev/$PROJECT/courier; ME=$(gcloud config get-value account)
API_URL=$(gcloud run services describe parcel-api --region $R --
format='value(status.url)')
WEB_URL=$(gcloud run services describe track-web --region $R --
format='value(status.url)')
TOKEN() { gcloud auth print-identity-token; }
```

## Task 0 — Baseline (not graded)

Run the baseline once. It enables the APIs, creates the courier repository and the two runtime identities, builds parcel-api 2.0 and track-web 1.0 with Cloud Build, and deploys them exactly as checkpoint 1 ended: parcel-api authenticated-only with DEPOT=BLR, track-web public and allowed to invoke it. It takes about five minutes and is safe to re-run.

### CLOUD SHELL

```
bash baseline.sh
# ends with: parcel-api anonymous: HTTP 403 · {"service":"parcel-api","version":"2.0",
...} · status: <b>out for delivery · baseline done
```

If the last line is missing, read the error, fix the cause (usually an API still enabling) and run it again. Do not start Task 1 until the baseline check passes.

## Task 1 — Secrets and identities

| Suggested time | Points | Produces                                                                                           |
|----------------|--------|----------------------------------------------------------------------------------------------------|
| 30 min         | 30     | a secret with two versions, a revision pinned to version 2, four bindings, one new service account |

### GOAL

Move the API key into Secret Manager the way the IAM session taught: created as a secret, mounted into parcel-api as a file pinned to a version, rotated by adding a version and moving the pin, the old version disabled. Grant access at the smallest scope. Then create the identity the pipeline will deploy as, with exactly the rights it needs.

### REQUIREMENTS

- Secret courier-api-key, automatic replication, version 1 = any 32-character value (openssl rand -hex 16 is fine).
- parcel-api-sa may read that secret and nothing else: roles/secretmanager.secretAccessor granted ON THE SECRET, not on the project.
- parcel-api mounts the secret as the file /secrets/api-key, pinned to a version — never latest. Prove it with GET /config (authenticated): secret\_file.present true, length 32.
- Rotate: add version 2, deploy a revision pinned to version 2, confirm the sha256\_8 in /config changed, then disable version 1 (do not destroy it).
- Service account courier-deployer with roles/run.developer and roles/clouddeploy.jobRunner on the project, and roles/iam.serviceAccountUser granted ON parcel-api-sa (the account resource), not on the project.

### ACCEPTANCE CRITERIA — WHAT THE GRADER CHECKS

- gcloud secrets versions list courier-api-key → 1 DISABLED, 2 ENABLED.
- parcel-api's template has a volume from courier-api-key with key "2" (gcloud run services describe ... --format='yaml(spec.template.spec.volumes)').
- The secret's IAM policy names parcel-api-sa as secretAccessor; the project's policy does not.
- courier-deployer holds run.developer and clouddeploy.jobRunner on the project; parcel-api-sa's own IAM policy names courier-deployer as serviceAccountUser; the project's policy does not.

### ALLOWED HINTS

- --update-secrets=/secrets/api-key=courier-api-key:1 mounts a file; a name without a leading slash would make an env var. --tag v1 on the revision makes it easy to find.
- gcloud iam service-accounts add-iam-policy-binding <account> --member ... --role roles/iam.serviceAccountUser is the account-level grant.
- The revision stores a reference, not the value — a describe never shows the key.

## Task 2 — Pipeline and promotion

| Suggested time | Points | Produces                                                                                      |
|----------------|--------|-----------------------------------------------------------------------------------------------|
| 40 min         | 40     | one green build, one image tagged by commit, one pipeline, one release promoted to production |

### GOAL

Build parcel-api the way the CI/CD session did — one Cloud Build run whose gates (lint, unit tests, dependency audit, integration check, vulnerability scan) must all pass before an image tagged with the commit is pushed — then register a Cloud Deploy pipeline and promote that exact image by digest: staging automatically, production behind your approval and a 20 % canary that you advance.

### REQUIREMENTS

- A git commit in the kit folder so the build has a short SHA to tag by. Run cloudbuild.yaml with --substitutions=\_SHA=<short sha>. All ten steps green.
- Fill your project id into the three deploy manifests (sed PROJECT\_ID → your project), then gcloud deploy apply the pipeline and its two targets.
- A release named r-<sha> created from deploy/ with --images=parcel-api=<image by DIGEST>. Staging deploys on its own: parcel-api-staging answers with depot STG.
- Promote to prod, approve the rollout, observe the canary (20 % on the new revision), then advance it to 100 %.
- Afterwards track-web must still work and /config must still show the secret: the pipeline's manifest carries the identity, the secret mount and DEPOT=BLR; the invoker binding is IAM and survives.

### ACCEPTANCE CRITERIA — WHAT THE GRADER CHECKS

- A SUCCESS build with a real \_SHA whose steps include lint, unit-tests, dependency-scan and vulnerability-scan.
- Image courier/parcel-api:<sha> exists in Artifact Registry.
- Pipeline parcel-api with stages staging then prod; prod requires approval and executes as courier-deployer.
- Release r-<sha>: staging rollout SUCCEEDED; prod rollout APPROVED and SUCCEEDED.
- parcel-api-staging exists and is not public; a parcel-api revision runs the release digest.

### ALLOWED HINTS

- The build takes five to six minutes; read Task 3 while it runs.
- gcloud artifacts docker images describe \$AR/parcel-api:\$SHA --format='value(image\_summary.digest)' gives the digest.
- gcloud deploy rollouts list --release r-<sha> ... shows state and approvalState per target; the prod rollout's name is what approve and advance take.
- If a rollout fails at DEPLOY, the log in Cloud Deploy → rollout names the missing permission — it is always one of the three from Task 1.

## Task 3 — Signals

| Suggested time | Points | Produces                                              |
|----------------|--------|-------------------------------------------------------|
| 30 min         | 30     | one metric, one channel, one policy, one uptime check |

### GOAL

Give parcel-api the three signals the observability session built: a log-based metric on the application's own fault line, an alert on 5xx that reaches an email address, and an uptime check on the public page. Generate a minute of faults first so the metric has something to count.

### REQUIREMENTS

- Faults to count: set `FAULT_RATE=0.3` on parcel-api, send 40 or so page loads through track-web, then remove the setting. (If traffic is pinned to a named revision after Task 2, follow each update with `update-traffic --to-latest`.)
- Log-based counter metric `parcel_api_faults` on: `resource.type="cloud_run_revision" AND resource.labels.service_name="parcel-api" AND textPayload:"simulated fault"`.
- An email notification channel to your own address, and the alert policy from `observability/alert-5xx.json` with `CHANNEL_NAME` replaced by the channel's resource name.
- An uptime check on track-web's host, path `/healthz`, every minute.

### ACCEPTANCE CRITERIA — WHAT THE GRADER CHECKS

- gcloud logging metrics describe `parcel_api_faults` shows the filter above (service name and the phrase).
- An email channel exists; a policy whose display name mentions parcel-api and 5xx exists and lists a notification channel.
- gcloud monitoring uptime list-configs shows a check whose host is the track-web URL and whose path is `/healthz`.

### ALLOWED HINTS

- `gcloud beta monitoring channels create --display-name ... --type=email --channel-labels=email_address=$ME --format='value(name)'` returns the name the JSON needs.
- The uptime check wants the host only: `${WEB_URL#https://}`.
- Logs take up to a minute to land; the metric counts from creation onward, not retroactively.

## Submission

- Fill SUBMISSION.md — every blank, real values from your project. Hand it in as a file or paste it into the programme's form.
- Leave everything running: secret, services, pipeline, policies. The instructor grades from live state and tears down afterwards.
- Do not run any teardown.

### CHECKLIST BEFORE YOU STOP

- Secret versions: 1 disabled, 2 enabled; parcel-api pinned to 2.
- Prod rollout SUCCEEDED; parcel-api-staging exists.
- Metric, channel, policy, uptime check all present under the fixed names.
- SUBMISSION.md complete — no \_\_\_\_\_ left.